
ОРГАНІЗАЦІЯ БАЗ ДАНИХ ТА ЗНАНЬ

Лекція 10

МОВА SQL. ФОРМУВАННЯ
ЗАПИТІВ ДО БАЗИ ДАНИХ.

Мова Структурованих Запитів (SQL - Structured Query Language) розроблена корпорацією IBM на початку 1970-х років.

Поточна версія стандарту мови SQL прийнята в 1992 р. Офіційна назва стандарту - Міжнародний стандарт мови баз даних SQL 1992 (International Standard Database Language SQL), неофіційна назва - SQL / 92, або SQL-92, або SQL2. Документ, що описує стандарт, містить понад 600 сторінок.

SQL - це потужна і в той же час нескладна мова для управління базами даних. Вона підтримується практично всіма сучасними базами даних.

SQL підрозділяється на дві підмножини команд: **DDL** (Data Definition Language - мова визначення даних) і **DML** (Data Manipulation Language - мова обробки даних). Команди DDL використовуються для створення нових баз даних, таблиць і стовпців, а команди DML - для читання, запису, сортування, фільтрування, видалення даних.

Оператори SQL

Оператори DDL (Data Definition Language) - оператори визначення об'єктів бази даних:

- **CREATE SCHEMA** - створити схему бази даних;
- **DROP SCHEMA** - видалити схему бази даних;
- **CREATE TABLE** - створити таблицю;
- **ALTER TABLE** - змінити таблицю;
- **DROP TABLE** - видалити таблицю;
- **CREATE DOMAIN** - створити домен;

Оператори SQL

- **ALTER DOMAIN** - змінити домен;
- **DROP DOMAIN** - видалити домен;
- **CREATE COLLATION** - створити послідовність;
- **DROP COLLATION** - видалити послідовність;
- **CREATE VIEW** - створити уявлення;
- **DROP VIEW** - видалити уявлення.

Оператори SQL

Оператори DML (Data Manipulation Language) - оператори маніпулювання даними:

- **SELECT** - відібрати рядки з таблиць;
- **INSERT** - додати рядки в таблицю;
- **UPDATE** - змінити рядки в таблиці;
- **DELETE** - видалити рядки в таблиці;
- **COMMIT** - зафіксувати внесені зміни;
- **ROLLBACK** - відкотити внесені зміни.

Оператори SQL

Оператори захисту і управління даними:

- **CREATE ASSERTION** - створити обмеження;
- **DROP ASSERTION** - видалити обмеження;
- **GRANT** - надати привілеї користувачу або додатку на маніпулювання об'єктами;
- **REVOKE** - скасувати привілеї користувача або додатки.

Приклади використання оператора **INSERT**

Приклад 1. Вставка одного рядка в таблицю:
INSERT INTO
P (PNUM, PNAME)
VALUES (4, "Іванов");

Приклад 2. Вставка в таблицю кількох рядків, обраних з іншої таблиці (в таблицю TMP_TABLE вставляються дані про постачальників з таблиці P, мають номери, великі 2):

INSERT INTO
TMP_TABLE (PNUM, PNAME)
SELECT PNUM, PNAME
FROM P
WHERE P.PNUM > 2;

Приклади використання операторів **UPDATE** і **DELETE**

Приклад 3. Оновлення декількох рядків в таблиці:

UPDATE P

SET PNAME = "Пушник"

WHERE P.PNUM = 1;

Приклад 4. Видалення кількох рядків у таблиці:

DELETE FROM P

WHERE P.PNUM = 1;

Приклад 5. Видалення всіх рядків в таблиці:

DELETE FROM P;

Оператор **SELECT**

Оператор **SELECT** є фактично найважливішим для користувача і найскладнішим оператором SQL. Він призначений для вибірки даних з таблиць, тобто він, власне, і реалізує одне з основних призначень бази даних - надавати інформацію користувачеві.

Оператор **SELECT**

Оператор **SELECT** завжди виконується над деякими таблицями, що входять в базу даних.

Результатом виконання оператора **SELECT** завжди є таблиця. Таким чином, за результатами дій оператор **SELECT** схожий на оператори реляційної алгебри.

Приклади використання оператора **SELECT**

Відбір даних з однієї таблиці

Приклад 6. Вибрати всі дані з таблиці постачальників (**ключові слова SELECT FROM**):

```
SELECT *  
FROM P;
```

Приклад 7. Вибрати всі рядки з таблиці постачальників, які задовольняють деякому умові (**ключове слово WHERE**):

```
SELECT *  
FROM P  
WHERE P.PNUM > 2;
```

Приклади використання оператора **SELECT**

Відбір даних з однієї таблиці

Приклад 8. Вибрати деякі колонки з вихідної таблиці (вказівка списку відбираються колонок):

```
SELECT P.NAME  
FROM P;
```

Приклад 9. Вибрати деякі колонки з вихідної таблиці, видаливши з результату повторювані рядки (ключове слово **DISTINCT**):

```
SELECT DISTINCT P.NAME  
FROM P;
```

Приклад 10. Використання скалярних виразів і перейменувань колонок в запитах (ключове слово **AS**):

SELECT

```
    TOVAR.TNAME,  
    TOVAR.KOL,  
    TOVAR.PRICE,  
    "=" AS EQU,  
    TOVAR.KOL * TOVAR.PRICE AS SUMMA  
FROM TOVAR;
```

TNAME	KOL	PRICE	EQU	SUMMA
Болт	10	100	=	1000
гайка	20	200	=	4000
гвинт	30	300	=	9000

Приклад 11. Впорядкування результатів запиту
(ключове слово **ORDER BY**):

SELECT

PD.PNUM,

PD.DNUM,

PD.VOLUME

FROM PD

ORDER BY DNUM;

PNUM	DNUM	VOLUME
1	1	100
2	1	150
3	1	1000
1	2	200
2	2	250
1	3	300

Приклад 12. Впорядкування результатів запиту по декількох полях зі зростанням або убутанням (ключові слова **ASC, DESC**):

SELECT

```
PD.PNUM,  
PD.DNUM,  
PD.VOLUME  
FROM PD  
ORDER BY  
DNUM ASC,  
VOLUME DESC;
```

PNUM	DNUM	VOLUME
3	1	1000
2	1	150
1	1	100
2	2	250
1	2	200
1	3	300

Приклади використання оператора SELECT

Відбір даних з декількох таблиць

Приклад 13. Природне з'єднання таблиць (спосіб 1 - явне вказівка умов з'єднання):

```
SELECT  
  P.PNUM,  
  P.PNAME,  
  PD.DNUM,  
  PD.VOLUME  
FROM P, PD  
WHERE P.PNUM = PD.PNUM;
```

PNUM	PNAME	DNUM	VOLUME
1	Іванов	1	100
1	Іванов	2	200
1	Іванов	3	300
2	Петров	1	150
2	Петров	2	250
3	Сидоров	1	1000

Приклад 14. Природне з'єднання таблиць
(спосіб 2 - ключові слова JOIN USING):

```
SELECT  
    P.PNUM,  
    P.PNAME,  
    PD.DNUM,  
    PD.VOLUME  
FROM P JOIN PD USING PNUM;
```

Ключове слово **USING** дозволяє явно вказати,
за якими із загальних колонок таблиць буде
проводитися з'єднання.

Приклад 15. Природне з'єднання таблиць (спосіб 3 - ключове слово NATURAL JOIN):

```
SELECT  
    P.PNUM,  
    P.PNAME,  
    PD.DNUM,  
    PD.VOLUME  
FROM P NATURAL JOIN PD;
```

У розділі FROM не вказано, по яких полях здійснюється з'єднання. **NATURAL JOIN** автоматично з'єднує по всім однаковим полям у таблицях.

Приклад 16. Природне з'єднання трьох таблиць:

SELECT

P.PNAME,

D.DNAME,

PD.VOLUME

FROM

P NATURAL JOIN PD NATURAL JOIN D;

PNAME	DNAME	VOLUME
Іванов	Болт	100
Іванов	гайка	200
Іванов	гвинт	300
Петров	Болт	150
Петров	гайка	250
Сидоров	Болт	1000

Приклад 17. Прямий добуток таблиць:

```
SELECT  
P.PNUM,  
P.PNAME,  
D.DNUM,  
D.DNAME  
FROM P, D;
```

PNUM	PNAME	DNUM	DNAME
1	Іванов	1	Болт
1	Іванов	2	гайка
1	Іванов	3	гвинт
2	Петров	1	Болт
2	Петров	2	гайка
2	Петров	3	гвинт
3	Сидоров	1	Болт
3	Сидоров	2	гайка
3	Сидоров	3	гвинт

Приклад 18. З'єднання таблиць за довільними умовами.

Таблиця 10.1. Відношення (таблиця) Р (Постачальники)

PNUM	PNAME	PSTATUS
1	Іванов	4
2	Петров	1
3	Сидоров	2

Таблиця 10.2. Відношення (таблиця) D (Деталі)

DNUM	DNAME	DSTATUS
1	Болт	3
2	гайка	2
3	гвинт	1



PNUM	PNAME	PSTATUS	DNUM	DNAME	DSTATUS
1	Іванов	4	1	Болт	3
1	Іванов	4	2	гайка	2
1	Іванов	4	3	гвинт	1
2	Петров	1	3	гвинт	1
3	Сидоров	2	2	гайка	2
3	Сидоров	2	3	гвинт	1

SELECT
P.PNUM,
P.PNAME,
P.PSTATUS,
D.DNUM,
D.DNAME,
D.DSTATUS
FROM P, D
WHERE P.PSTATUS > =
D.DSTATUS;

Використання імен кореляції (аліасів, псевдонімів)

Приклад 19. Відібрати всі пари постачальників таким чином, щоб перший постачальник в парі мав статус, більший статусу другої постачальника:

SELECT

P1.PNAME AS PNAME1,

P1.PSTATUS AS PSTATUS1,

P2.PNAME AS PNAME2,

P2.PSTATUS AS PSTATUS2

FROM

P P1, P P2

WHERE P1.PSTATUS1 > P2.PSTATUS2;

PNAME1	PSTATUS1	PNAME2	PSTATUS2
Іванов	4	Петров	1
Іванов	4	Сидоров	2
Сидоров	2	Петров	1

Приклад 20. Розглянемо ситуацію, коли деякі постачальники (назвемо їх контрагенти) можуть виступати як в якості постачальників деталей, так і в якості одержувачів.

Таблиця 10.3. Відношення (таблиця) CONTRAGENTS

номер контрагента NUM	Найменування контрагента NAME
1	Іванов
2	Петров
3	Сидоров

Таблиця 10.4. Відношення (таблиця) DETAILS (Деталі)

номер деталі DNUM	Найменування деталі DNAME
1	Болт
2	гайка
3	гвинт

Таблиця 10.5. Відношення (таблиця) CD (Поставки)

номер поставщика PNUM	номер отримувача CNUM	номер деталі DNUM	Обладнання, що поставляється кількість VOLUME
1	2	1	100
1	3	2	200
1	3	3	300
2	3	1	150
2	3	2	250
3	1	1	1000

Відповідь на питання «хто кому що в якій кількості поставляє» дається наступним запитом:

SELECT

P.NAME AS PNAME,

C.NAME AS CNAME,

DETAILS.DNAME,

CD.VOLUME

FROM

CONTRAGENTS P,

CONTRAGENTS C,

DETAILS,

CD

WHERE

P.NUM = CD.PNUM AND

C.NUM = CD.CNUM AND

D.DNUM = CD.DNUM;

Найменування поставщика PNAME	Найменування отримувача CNAME	Найменування деталі DNAME	Обладнання, що поставляється кількість VOLUME
Іванов	Петров	Болт	100
Іванов	Сидоров	гайка	200
Іванов	Сидоров	гвинт	300
Петров	Сидоров	Болт	150
Петров	Сидоров	гайка	250
Сидоров	Іванов	Болт	1000

Використання агрегатних функцій в запитах

Приклад 21. Отримати загальну кількість постачальників (ключове слово COUNT):

**SELECT COUNT (*) AS N
FROM P;**

В результаті отримаємо таблицю з одним стовпцем і одним рядком, що містить кількість рядків з таблиці P:

N
3

Приклад 22. Отримати загальне, максимальне, мінімальне та середнє значення кількості деталей, що поставляються (ключові слова SUM, MAX, MIN, AVG):

SELECT
 SUM (PD.VOLUME) AS SM,
 MAX (PD.VOLUME) AS MX,
 MIN (PD.VOLUME) AS MN,
 AVG (PD.VOLUME) AS AV
FROM PD;

В результаті отримаємо наступну таблицю з одним рядком:

SM	MX	MN	AV
2000	1000	100	333.33333333

Використання агрегатних функцій з угрупованнями

Приклад 23. Для кожного постачальника отримати сумарну кількість деталей, що поставляється (ключове слово GROUP BY):

```
SELECT  
    PD.DNUM,  
    SUM (PD.VOLUME) AS SM  
GROUP BY PD.DNUM;
```

DNUM	SM
1	1250
2	450
3	300

Приклад 24. Отримати номери деталей, що поставляється сумарна кількість яких перевищує 400 (ключове слово HAVING):

```
SELECT  
    PD.DNUM,  
    SUM (PD.VOLUME) AS SM  
GROUP BY PD.DNUM  
HAVING SUM (PD.VOLUME)> 400;
```

Найменування поставщика PNAME	Найменування отримувача CNAME	Найменування деталі DNAME	Обладнання, що поставляється кількість VOLUME
Іванов	Петров	Болт	100
Іванов	Сидоров	гайка	200
Іванов	Сидоров	гвинт	300
Петров	Сидоров	Болт	150
Петров	Сидоров	гайка	250
Сидоров	Іванов	Болт	1000

Використання підзапитів

Приклад 25. Отримати список постачальників, статус яких менше максимального статусу в таблиці постачальників (порівняння з підзапитом):

```
SELECT *  
FROM P  
WHERE P.STATUS <  
      (SELECT MAX (P.STATUS)  
        FROM P);
```

PNAME1	PSTATUS1	PNAME2	PSTATUS2
Іванов	4	Петров	1
Іванов	4	Сидоров	2
Сидоров	2	Петров	1

Приклад 26. Використання предиката IN.
Отримати список постачальників, що
поставляють деталь номер 2:

```
SELECT *  
FROM P  
WHERE P.PNUM IN  
      (SELECT DISTINCT PD.PNUM  
        FROM PD  
        WHERE PD.DNUM = 2);
```

Таблиця 10.3. Відношення (таблиця) CONTRAGENTS

номер контрагента NUM	Найменування контрагента NAME
1	Іванов
2	Петров
3	Сидоров

Таблиця 10.4. Відношення (таблиця) DETAILS (Деталі)

номер деталі DNUM	Найменування деталі DNAME
1	Болт
2	гайка
3	гвинт

Таблиця 10.5. Відношення (таблиця) CD (Поставки)

номер поставщика PNUM	номер отримувача CNUM	номер деталі DNUM	Обладнання, що поставляється кількість VOLUME
1	2	1	100
1	3	2	200
1	3	3	300
2	3	1	150
2	3	2	250
3	1	1	1000

Результат виконання запиту буде еквівалентний результату наступної послідовності дій:

1. Виконати один раз вкладений підзапит і отримати список номерів постачальників, що поставляють деталь номер 2.
 2. Просканувати таблицю постачальників Р, щоразу перевіряючи, чи міститься номер постачальника в результаті підзапиту.
-

Приклад 27. Використання предиката EXIST.
Отримати список постачальників, що
поставляють деталь номер 2:

```
SELECT *  
FROM P  
WHERE EXIST  
  (SELECT *  
    FROM PD  
    WHERE  
      PD.PNUM = P.PNUM AND  
      PD.DNUM = 2);
```

Результат виконання запиту буде еквівалентний результату наступної послідовності дій:

1. Просканувати таблицю постачальників P , кожен раз виконуючи підзапит з новим значенням номера постачальника, узятим з таблиці P .
 2. В результат запиту включити тільки ті рядки з таблиці постачальників, для яких вкладений підзапит повернув непорожня множина рядків.
-

Приклад 28. Використання предиката NOT EXIST. Отримати список постачальників, які не постачають деталь номер 2:

```
SELECT *  
FROM P  
WHERE NOT EXIST  
  (SELECT *  
    FROM PD  
    WHERE  
      PD.PNUM = P.PNUM AND  
      PD.DNUM = 2);
```

Приклад 29. Отримати імена постачальників, що поставляють всі деталі:

```
SELECT DISTINCT PNAME  
FROM P  
WHERE NOT EXIST  
  (SELECT *  
    FROM D  
    WHERE NOT EXIST  
      (SELECT *  
        FROM PD  
        WHERE  
          PD.DNUM = D.DNUM AND  
          PD.PNUM = P.PNUM));
```

Використання об'єднання, перетину і різниці

Приклад 30. Отримати імена постачальників, що мають статус, більший 3 або постачають хоча б одну деталь номер 2 (об'єднання двох підзапитів - ключове слово UNION):

```
SELECT P.PNAME  
FROM P  
WHERE P.STATUS > 3  
UNION  
SELECT P.PNAME  
FROM P, PD  
WHERE P.PNUM = PD.PNUM AND  
PD.DNUM = 2;
```

Приклад 31. Отримати імена постачальників, що мають статус, більший 3 і одночасно поставляють хоча б одну деталь номер 2 (перетин двох підзапитів - ключове слово INTERSECT):

```
SELECT P.PNAME  
FROM P  
WHERE P.STATUS > 3  
INTERSECT  
SELECT P.PNAME  
FROM P, PD  
WHERE P.PNUM = PD.PNUM AND  
PD.DNUM = 2;
```

Приклад 32. Отримати імена постачальників, що мають статус, більший 3, за винятком тих, хто поставляє хоча б одну деталь номер 2 (різниця двох підзапитів - ключове слово EXCEPT):

```
SELECT P.PNAME  
FROM P  
WHERE P.STATUS > 3  
EXCEPT  
SELECT P.PNAME  
FROM P, PD  
WHERE P.PNUM = PD.PNUM AND  
PD.DNUM = 2;
```

PNAME1	PSTATUS1	PNAME2	PSTATUS2
Іванов	4	Петров	1
Іванов	4	Сидоров	2
Сидоров	2	Петров	1

Синтаксис оператора вибірки даних (SELECT)

нотація

- При описі синтаксису операторів звичайно використовуються умовні позначення, відомі як стандартні форми Бекуса-Наура (BNF). У BNF позначеннях використовуються наступні елементи:
1. Символ «:: =» означає рівність за визначенням. Зліва від знака коштує визначається поняття, праворуч - власне визначення поняття.
 2. Ключові слова записуються прописними буквами. Вони зарезервовані і складають частину оператора.
 3. Мітки-заповнювачі конкретних значень елементів і змінних записуються курсивом.

BNF- нотація

4. Необов'язкові елементи оператора укладені в квадратні дужки [].
 5. Вертикальна риса | вказує на те, що всі попередні їй елементи списку є необов'язковими і можуть бути замінені будь-яким іншим елементом списку після цієї риси.
 6. Фігурні дужки {} вказують на те, що все знаходиться всередині них є єдиним цілим.
 7. Три крапки « $\perp$ » означає, що попередня частина оператора може бути повторена будь-яку кількість разів.
-

BNF- нотація

8. Три крапки, всередині якого знаходиться кома "., ..» вказує, що попередня частина оператора, що складається з декількох елементів, розділених комами, може мати довільне число повторень. Кому не можна ставити після останнього елемента.

Зауваження: дана угода не входить в стандарт BNF, але дозволяє більш точно описати синтаксис операторів SQL.

9. Круглі дужки () є елементом оператора.

Синтаксис оператора вибірки

Select-вираз в розділі SELECT, що використовується в якості значення для відбирається стовпчика, має повертати таблицю, що складається з одного рядка і одного стовпця, тобто скалярний вираз.

Умовний вираз в розділі **WHERE** має обчислюватися для кожного рядка, що є кандидатом в результуюча множина рядків. В цьому умовному вираженні можна використовувати вкладені запити.

Розділ **HAVING** містить умовний вираз, що обчислюється для кожної групи, яка визначається списком угруповання в розділі GROUP BY.

Скалярний вираз - в якості скалярних виразів в розділі SELECT можуть виступати або імена стовпців таблиць, що входять в розділ FROM, або прості функції, які повертають скалярні значення.

функція агрегування:: =

COUNT(*) | {{COUNT | MAX | MIN | SUM | AVG}
([ALL | DISTINCT] Скалярний вираз)}

Конструктор значень таблиці:: =

VALUES Конструктор значень рядка., ..

Конструктор значень рядка:: =

елемент конструктора | (Елемент конструктора.,
..) | Select-вираз

Зауваження. Select-вираз, що

використовується в конструкторі значень рядка,
зобов'язана повертати рівно один рядок.

елемент конструктора:: =

Вираз для обчислення значення | NULL | DEFAULT

*поєднана таблиця:: =
перехресне з'єднання*

| Природне з'єднання | З'єднання за
допомогою предиката | З'єднання за
допомогою імен стовпців | з'єднання
об'єднання

Синтаксис з'єднаних таблиць

Тип з'єднання:: =

INNER

| LEFT [OUTER] | RIGHT [OUTER] | FULL [OUTER]

перехресне з'єднання:: =

Таблиця ACROSS JOIN Таблиця В

природне з'єднання:: =

Таблиця A[NATURAL] [Тип з'єднання] JOIN Таблиця В

З'єднання за допомогою предиката:: =

Таблиця A[Тип з'єднання] JOIN Таблиця В ON

Предикат

З'єднання за допомогою імен стовпців:: =

Таблиця A[Тип з'єднання] JOIN Таблиця В USING (Ім'я
стовпця., ..)

з'єднання об'єднання:: =

Таблиця AUNION JOIN Таблиця В

CROSS JOIN - Перехресне з'єднання повертає просто декартовій твір таблиць. Таке з'єднання в розділі FROM може бути замінено списком таблиць через кому.

NATURAL JOIN - Природне з'єднання проводиться за всіма стовпцями таблиць A і B, які мають однакові імена. У результуючу таблицю однакові стовпці вставляються тільки один раз.

JOIN **ON** - З'єднання за допомогою предиката з'єднує рядки таблиць A і B за допомогою вказаного предиката.

JOIN - **USING** - З'єднання за допомогою імен стовпців з'єднує відносини подібно до природного з'єднання по тим загальним стовпцях таблиць А і В, які вказані в списку USING.

OUTER - Ключове слово OUTER (зовнішній) не є обов'язковими, воно не використовується ні в яких операціях з даними.

INNER - Тип з'єднання «внутрішнє».
Внутрішній тип з'єднання використовується за умовчанням, коли тип явно не заданий. У таблицях А і В з'єднуються тільки ті рядки, для яких знайдено збіг.

LEFT (OUTER) - Тип з'єднання «ліве (зовнішнє)». Ліве з'єднання таблиць A і B включає в себе всі рядки з лівої таблиці A і ті рядки з правої таблиці B, для яких виявлено збіг. Для рядків з таблиці A, для яких не знайдено відповідності в таблиці B, в стовпці, що витягають із таблиці B, заносяться значення NULL.

RIGHT (OUTER) - Тип з'єднання «праве (зовнішнє)». Праве з'єднання таблиць A і B включає в себе всі рядки з правої таблиці B і ті рядки з лівої таблиці A, для яких виявлено збіг. Для рядків з таблиці B, для яких не знайдено відповідності в таблиці A, в стовпці, що витягають із таблиці A заносяться значення NULL.

FULL (OUTER) - Тип з'єднання «повне (зовнішнє)». Це комбінація лівого і правого з'єднань. В повне з'єднання включаються всі рядки з обох таблиць. Для співпадаючих рядків поля заповнюються реальними значеннями, для незбіжних рядків поля заповнюються відповідно до правил лівого і правого з'єднань.

UNION JOIN - З'єднання об'єднання є зворотним по відношенню до внутрішнього з'єднання. Воно включає тільки ті рядки з таблиць A і B, для яких не знайдено збігів. У них використовуються значення NULL для стовпців, отриманих з іншої таблиці.

Синтаксис умовних виразів розділу WHERE

умовний вираз:: = [(] [NOT] {Предикат порівняння | Предикат between | Предикат in | Предикат like | Предикат null | Предикат кількісного порівняння | Предикат exist | Предикат unique | Предикат match | Предикат overlaps} [{AND | OR} Умовний вираз] [)] [IS [NOT] {TRUE | FALSE | UNKNOWN}]

предикат порівняння:: =

Конструктор значень рядка{= | < | > | <= | >= | <>} *Конструктор значень рядка*

Приклад 33. Порівняння поля таблиці і скалярного значення:

POSTAV.VOLUME > 100

Приклад 34. Порівняння двох сконструйованих рядків:

(PD.PNUM, PD.DNUM) = (1, 25)

Цей приклад еквівалентний умовного виразу

PD.PNUM = 1 AND PD.DNUM = 25

предикат between:: =

Конструктор значень рядка[NOT] BETWEEN

Конструктор значень рядка AND Конструктор значень рядка

Приклад 35. PD.VOLUME BETWEEN 10 AND 100

предикат in:: =

Конструктор значень рядка[NOT] IN {(Select-вираз) | (Вираз для обчислення значення., ..)}

Приклад 36.

P.PNUM IN (1, 2, 3, 5)

предикат like:: =

Вираз для обчислення значення рядка-пошуку
[NOT] LIKE

Вираз для обчислення значення рядка-шаблону
[ESCAPE Символ]

Зауваження. Предикат LIKE здійснює пошук рядка-пошуку в рядку-шаблоні. У рядку-шаблоні дозволяється використовувати два трафаретних символи:

- Символ підкреслення «_» може використовуватися замість будь-якого одиничного символу в рядку-пошуку,
 - Символ відсотка "%" може замінювати набір будь-яких символів в рядку-пошуку (число символів в наборі може бути від 0 і більше).
-

Квантори **ANY і SOME** є синонімами і повністю взаємозамінні.

Якщо вказано один з кванторів **ANY і SOME**, то предикат кількісного порівняння повертає **TRUE**, якщо порівнюване значення збігається хоча б з одним значенням, повернутому в підзапиті (select-вираженні).

Якщо вказано квантор **ALL**, то предикат кількісного порівняння повертає **TRUE**, якщо порівнюване значення збігається з кожним значенням, що повертається в підзапиті (select-вираженні).

Приклад 37.

```
P.PNUM = SOME (SELECT PD.PNUM FROM PD  
WHERE PD.DNUM = 2)
```

предикат exist:: =
EXIST(Select-вираз)

Зауваження. Предикат EXIST повертає значення TRUE, якщо результат підзапиту (select-вирази) не пустили.

предикат unique:: =
UNIQUE(Select-вираз)

зауваження. Предикат UNIQUE повертає TRUE, якщо в результаті підзапиту (select-вирази) немає співпадаючих рядків.

предикат match:: =

~~Конструктор значень рядка MATCH [UNIQUE]~~
[PARTIAL | FULL] (Select-вираз)

Предикат MATCH перевіряє, чи буде значення, визначене в конструкторі рядка збігатися зі значенням будь-якого рядка, отриманої в результаті підзапиту.

предикат overlaps:: =

Конструктор значень рядка OVERLAPS

Конструктор значень рядка

Предикат OVERLAPS, є спеціалізованим предикатом, що дозволяє визначити, чи буде вказаний період часу перекривати інший період часу.

Порядок виконання оператора SELECT

Стадія 1. Виконання одиночного оператора SELECT

Якщо в операторі присутні ключові слова UNION, EXCEPT і INTERSECT, то запит розбивається на кілька незалежних запитів, кожний з яких виконується окремо:

Крок 1 (*FROM*). Обчислюється пряме декартовій твір всіх таблиць, зазначених в обов'язковому розділі FROM. В результаті кроку 1 отримуємо таблицю A.

Крок 2 (WHERE). Якщо в операторі SELECT присутній розділ WHERE, то сканується таблиця A, отримана при виконанні кроку 1. При цьому для кожного рядка з таблиці A обчислюється умовний вираз, наведене в розділі WHERE. Тільки ті рядки, для яких умовний вираз повертає значення TRUE, включаються в результат. Якщо розділ WHERE опущений, то відразу переходимо до кроку 3. Якщо в умовному виразі беруть участь вкладені підзапити, то вони обчислюються відповідно до даної концептуальної схеми. В результаті кроку 2 отримуємо таблицю B.

Крок 3 (GROUP BY). Якщо в операторі SELECT присутній розділ GROUP BY, то рядки таблиці В, отриманої на другому кроці, групуються відповідно до списку угруповання, наведеними в розділі GROUP BY. Якщо розділ GROUP BY опущений, то відразу переходимо до кроку 4. В результаті кроку 3 отримуємо таблицю С.

Крок 4 (HAVING). Якщо в операторі SELECT присутній розділ HAVING, то групи, що не задовольняють умовному виразу, наведеного в розділі HAVING, виключаються. Якщо розділ HAVING опущений, то відразу переходимо до кроку 5. В результаті кроку 4 одержуємо таблицю D.

Крок 5 (*SELECT*). Кожна група, отримана на кроці 4, генерує один рядок результату в такий спосіб. Обчислюються всі скалярні вирази, зазначені в розділі *SELECT*. За правилами використання розділу *GROUP BY*, такі скалярні вирази повинні бути однаковими для всіх рядків усередині кожної групи. Для кожної групи обчислюються значення агрегатних функцій, наведених в розділі *SELECT*. Якщо розділ *GROUP BY* був відсутній, але в розділі *SELECT* є агрегатні функції, то вважається, що є всього одна група. Якщо немає ні розділу *GROUP BY*, ні агрегатних функцій, то вважається, що є стільки груп, скільки рядків відібрано до даного моменту. В результаті кроку 5 отримуємо таблицю *E*, яка містить стільки колонок, скільки елементів наведено в розділі *SELECT* і стільки рядків, скільки відібрано груп.

Стадія 2. Виконання операцій **UNION, **EXCEPT**, **INTERSECT****

Якщо в операторі SELECT присутні ключові слова UNION, EXCEPT і INTERSECT, то таблиці, отримані в результаті виконання 1-ї стадії, об'єднуються, віднімаються або перетинаються.

Стадія 3. Впорядкування результату

Якщо в операторі `SELECT` присутній розділ `ORDER BY`, то рядки отриманої на попередніх етапах таблиці упорядковуються відповідно до списку упорядкування, наведеному в розділі `ORDER BY`.

Насправді в РСКБД є оптимізатор, функцією якого є знаходження такого оптимального алгоритму виконання запиту, який гарантує отримання правильного результату.

Схематично **роботу оптимізатора** можна представити у вигляді послідовності декількох кроків:

Крок 1 (*Синтаксичний аналіз*). Поступив запит піддається синтаксичному аналізу. На цьому кроці визначається, чи правильно взагалі (з точки зору синтаксису SQL) сформульований запит. В ході розбору виробляється деякий внутрішньо уявлення запиту, що використовується на наступних кроках.

Крок 2 (Перетворення в канонічну форму).

Запит у внутрішньому поданні піддається ~~перетворенню в деяку канонічну форму~~. При перетворенні до канонічної форми використовуються як синтаксичні, так і семантичні перетворення. **Синтаксичні перетворення** (наприклад, приведення логічних виразів до кон'юнктивної або диз'юнктивної нормальній формі, заміна виразів « $x \text{ AND NOT } x$ » на «FALSE», і т.п.) дозволяють отримати нове внутрішньо уявлення запиту, синтаксично еквівалентну вихідного, але стандартне в деякому сенсі. В результаті семантичних перетворень виходить запит, синтаксично еквівалентний вихідному, але дає той же самий результат.

Крок 3 (Генерація планів виконання запиту і вибір оптимального плану). На цьому кроці оптимізатор генерує безліч можливих планів виконання запиту. Кожен план будується як комбінація низькорівневих процедур доступу до даних з таблиць, методам з'єднання таблиць. З усіх згенерованих планів вибирається план, що володіє мінімальною вартістю. При цьому аналізуються дані про наявність індексів у таблиць, статистичних даних про розподіл значень в таблицях, і т.п.

Крок 4. (Виконання плану запиту). На цьому кроці план, обраний на попередньому етапі, передається на реальне виконання.
